



DEPARTMENT OF ENGINEERING CYBERNETICS

TTK4145 - REAL TIME PROGRAMMING

Three Elevator System

Author:

Phillip-André Broch-Torp
Markus Jørgensen
Beimnet Zeru

1st May 2025

Table of Contents

1	Design	1
1.1	Networking	1
1.2	Order Manager	1
1.3	Master slave distributor	1
1.4	Further improvement	1
2	Case study	2
2.1	Order Addition & Deletion	2
2.2	Fault Tolerance	2
3	Figures	3

1 Design

The overall design ended up being much like that presented in the preliminary design report. The main changes were that the ordermanager handles all orders, the slave does not directly handle cab calls. In addition, only UDP is used.

1.1 Networking

The networking module is the module that sends and receives information between elevators. This is done through broadcasting on a port with UDP. The module is the same for all elevators, however, some parts of the code are only active for slaves, and some are only active for the master. The module fetches relevant information and constructs outgoing messages. In addition, it distributes information from incoming messages to the relevant modules. The information flow from and to the networking module can be seen in figure 1 and 2

1.2 Order Manager

The order manager is mainly active for the master, and handles the delegation of orders. Diagram 3 shows the order flow. The addition of new orders is done through a simple counter that increases on every button press, where every elevator has its own order counter. New orders are only acknowledged by the master if the incoming order counter is larger than what the master already has saved. The deletion of orders is all done through the master. The other elevators broadcasts their worldview, and the master will delete orders from all active orders, based on the elevator's state. Orders are free to be redistributed at any time, based on the state of the active elevators. That way, orders from elevators that enter inactive state, either through loss of motor voltage, long obstruction etc., will be redistributed automatically.

1.3 Master slave distributor

The master slave distributor (MSD) keeps track of active and alive elevators, and who the current master is. This means a major part of our fault tolerance happens within this module. Different timers will infer different failure modes, as displayed in the decision nodes in figure 4. When there are multiple masters, the master(s) stepping down will add all their current orders to requests to make sure the actual master can distribute these orders. This ensures that no orders are lost. When the current master fails, the backup process will start, and a new master will be elected. The saved orders will be applied to the new master, so that the orders will be distributed again, and no orders will be lost. This also ensures that cab orders for the dead elevator will be saved and will be given back when they come online again. If a disconnected elevator reconnects, it will get all its cab orders back, even if they have already been serviced. This will hurt performance, but make sure no orders are lost.

1.4 Further improvement

The system has an edge case when hall up and down are the only active orders on the whole system. They are taken by two elevators, but both are being cleared when the first elevator arrives at the floor, and not cleared separately when each elevator reaches the floor. This problem was addressed by a timer that blocked deletion of orders in the other direction for three seconds if the elevator detects that it is in the edge case, however the solution only works for a single elevator. A quick fix for this problem, could be to add an edge case detection to the order manager, prohibiting the system from re-delegating the orders. A long-term solution, ensuring robustness in the system for new undiscovered bugs, would be to use TCP for order deletion, and delete orders on door close, instead of on door open.

The system would not work above (80-90)% packet loss. The system gets busy trying to set master, resolve multiple masters, and no orders are being taken reliably. Therefore, a future improvement would be to count the average message received compared to our sending frequency and enter single elevator mode if the system estimates that the packet loss is high.

2 Case study

2.1 Order Addition & Deletion

In our distributed elevator system, a core challenge was ensuring concurrency with regard to orders. This being a challenge, as we want to remove orders based on triggering events in the single elevators, while also adding orders from the master to the single elevators. Removing orders directly in the single elevators would pose problems in determining if an additional order arriving from the master was a new order or an already completed order.

To combat this issue, we considered two approaches. One being the chosen approach, detailed in the design portion of this document and in figure 3. The other being a TCP-based approach, where order-altering events would trigger specific messages sent between the master and slaves, e.g., "delete this order", "add this order", and "new request".

The TCP-based approach ensures that we know all messages arrive through a handshake. This would limit the risk of unforeseen bugs, caused by our solutions to TCP functionality in a UDP system. It also uses less bandwidth than our chosen approach, as messages are sent only when needed, not periodically. However, it would force us to implement an additional messaging protocol, as we were already using UDP for heartbeats. Furthermore, concerns were raised with regard to packet loss, as simulated packet loss of 50% would give low chances of successful handshakes.

We decided to go for our chosen approach, as it would allow the system to stay operational in high packet loss simulations and did not force the implementation of an additional messaging protocol. Through testing, we uncovered that the current implementation was within the available bandwidth, meaning it was a non-issue for this project. We did, however, encounter some issues with a pure UDP approach, as mentioned in the further improvements section. For a real implementation, a combined UDP, TCP protocol could therefore be considered to ensure more robustness, lower energy consumption, and lower bandwidth usage.

2.2 Fault Tolerance

A major source of complexity in our implementation came from how we dealt with fault tolerance. Initially, we only considered the case of disconnecting or shut down elevators and their reintroduction onto a functioning, stable network. We considered two main approaches; One being the approach chosen, detailed in the design portion of this document. That approach has to consider all triggering events at all times and adds complexity when deciding what should happen based on the event. The other being a state machine approach, where specific events would only trigger in given states, as shown in figure 4.

Although the state machine approach seemed more flexible and understandable, it also came with an increased initial implementation time and complexity. With time-limited resources, we opted for the first option, as it seemed like the implementation with the least complexity and implementation time, while still fulfilling the system specifications and restrictions.

After our initial implementation, we later found multiple edge cases when elevators were reintroduced to imperfect systems. This forced additional implementation cost and complexity, through timers and other logic, at times, hurting the readability of the code. At that point we fell for the sunk cost fallacy, continuing with our initial approach, even though it became difficult to maintain and expand. In the end we also had to refactor the code to make it manageable, which probably took more time than just implementing the state machine in the first place.

If we were to redo the project, we should have taken more time researching possible edge cases, to better understand the actual complexity of the problem at hand. That way, we could have taken a more informed decision and probably ended up with the state machine approach. Generally, we also discovered that sub-problems have more complexity than what it seems initially. We should therefore utilize approaches that are easier to maintain and expand, to ensure flexibility when dealing with unforeseen problems, even if the initial implementation comes at some additional cost.

In order to implement the state machine approach, we would utilize the approach explained in figure 4. That way, we allow for five distinct MSD states that all have well-defined events for state change, and we don't have to consider them in states where they are not triggering events.

3 Figures

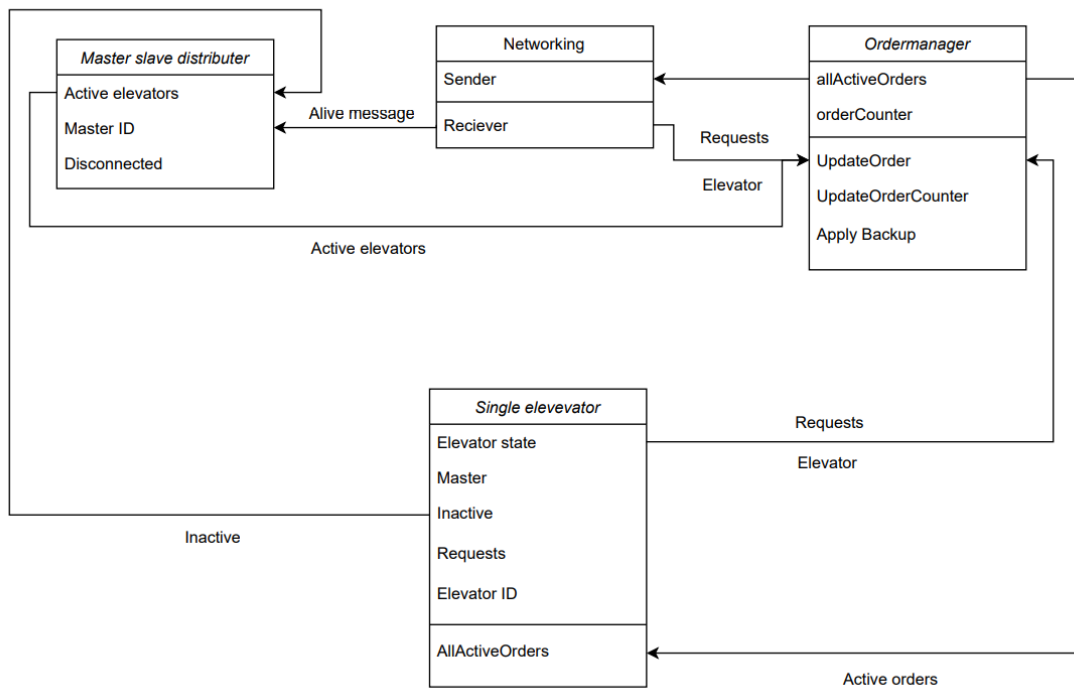


Figure 1: Class diagram for master operation

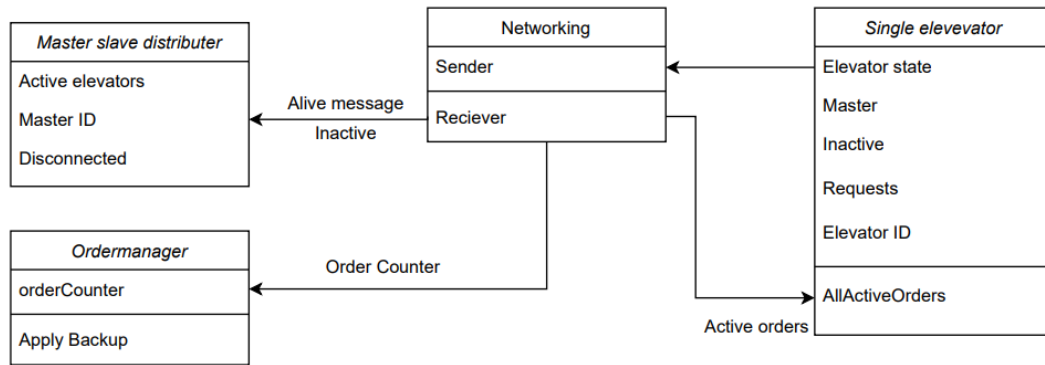


Figure 2: Class diagram for slave operation

Order flow

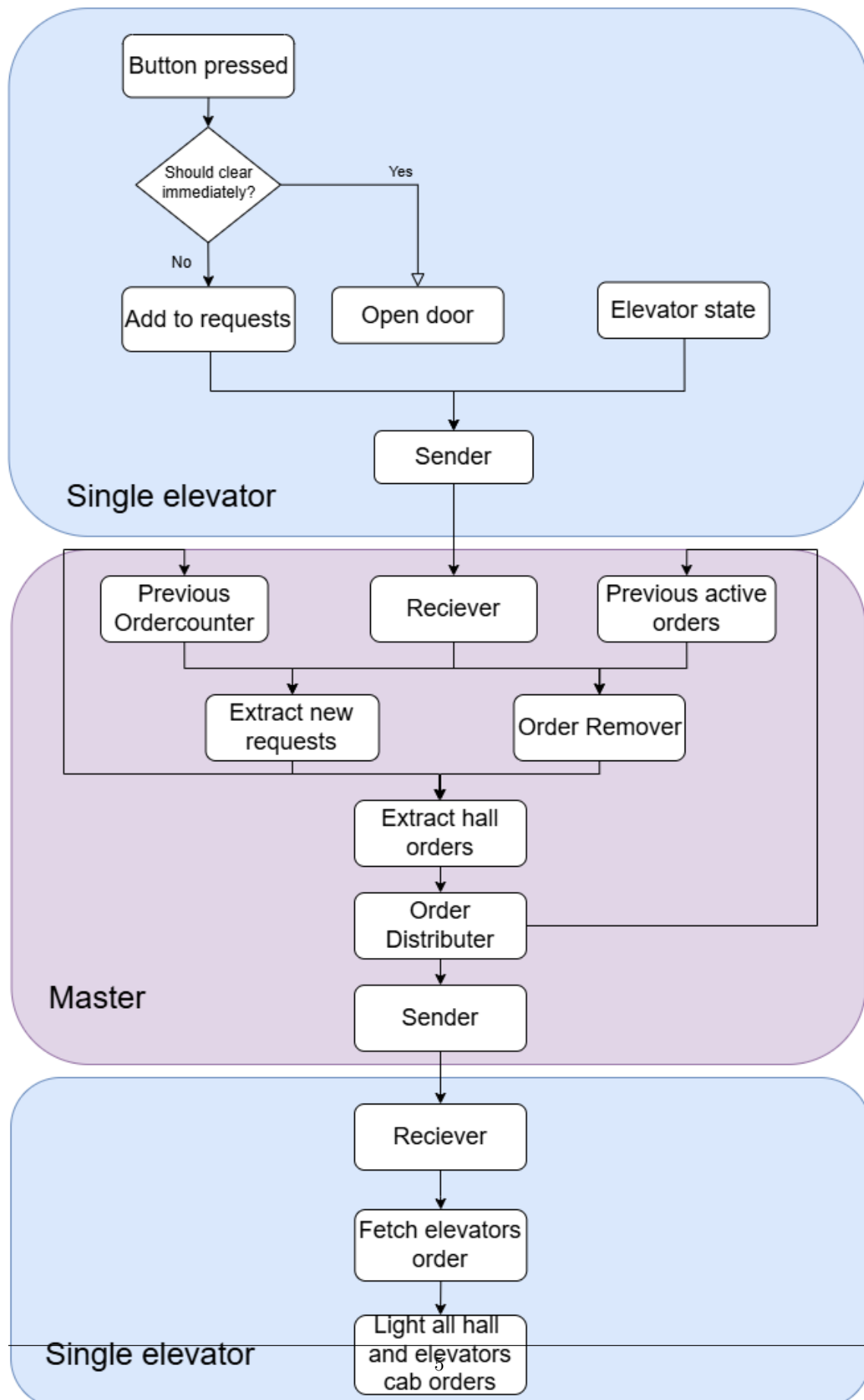


Figure 3: Flow chart for order handling

Master slave state diagram

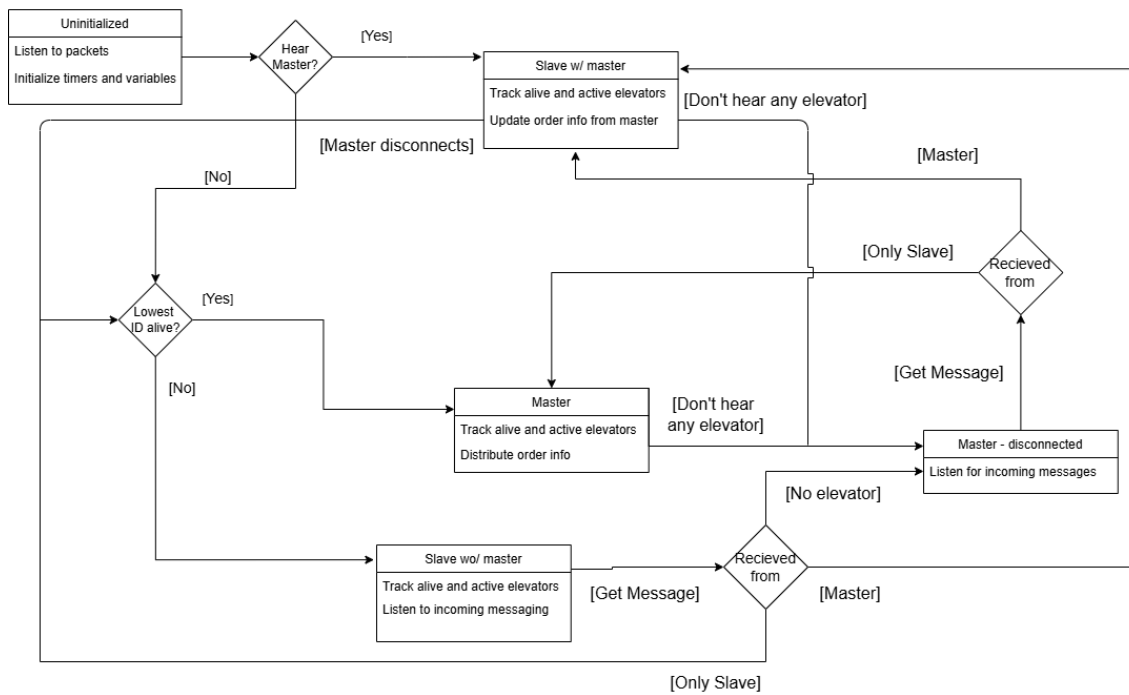


Figure 4: State machine diagram for master slave distributor